# Web3Geeks Support-Ticket Triage Agent

*Executive Report — Week 2 Capstone: Production-Ready Agent System, Evaluation & Deployment*

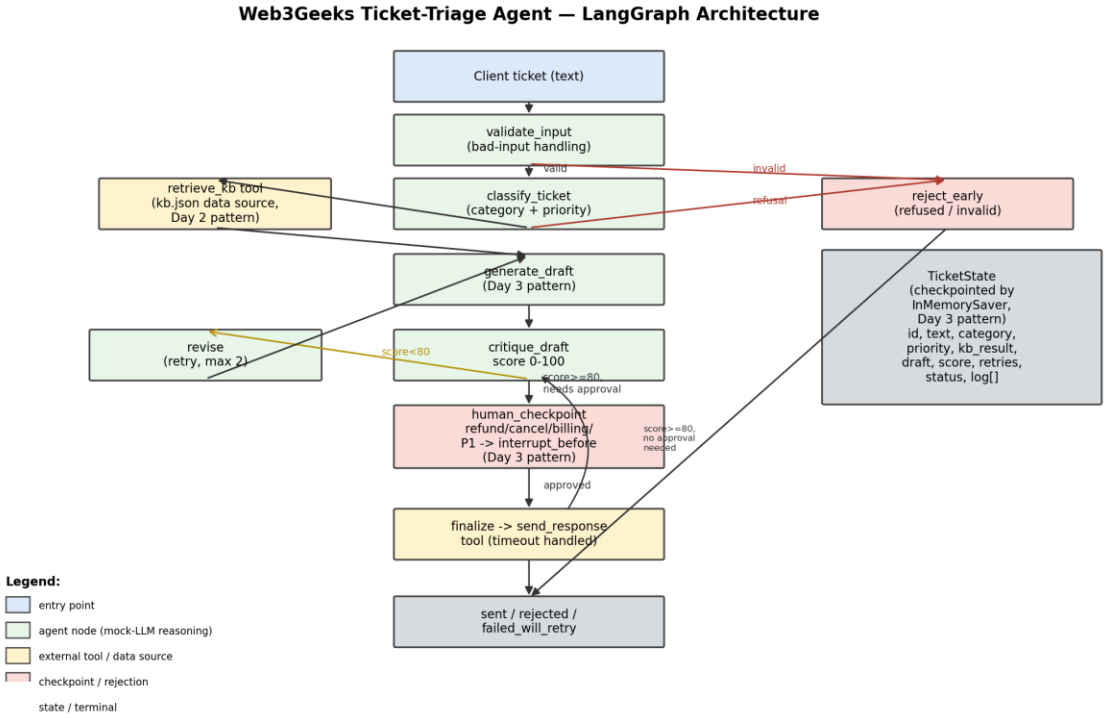
## 1. Business Goal

Web3Geeks' support inbox mixes routine questions (timelines, onboarding steps) with consequential requests (refunds, cancellations, billing disputes, smart-contract bug reports) that carry financial or contractual risk. The goal is to triage tickets automatically — classify, answer from a knowledge base, and draft a reply — while guaranteeing that anything consequential is reviewed by a human before it goes out, cutting first-response time on routine tickets without adding refund/cancellation risk.

## 2. Architecture

The agent is a single LangGraph state machine: validate_input -> classify_ticket -> retrieve_kb -> generate_draft -> critique_draft -> (revise and loop back if the critique score is below 80, up to 2 retries) -> human_checkpoint (for refunds, cancellations, billing disputes, contract-bug reports, and P1-critical tickets) -> finalize (dispatch). Invalid input and unsafe/out-of-scope requests exit early through a reject_early node. All state (ticket text, category, priority, KB match, draft, critique score, retries, status, and a per-node event log) lives in one typed State object that flows through every node and is checkpointed, which is what a monitoring layer reads.

This system builds directly on earlier work this week rather than starting from scratch: the KB-lookup tool follows the local-JSON-"database" tool pattern built on Day 2; the generate -> critique -> revise self-correction cycle and the interrupt_before + checkpointer mechanism for the human-approval gate are reused essentially as built on Day 3 (there, gating an email send; here, gating a ticket response); and the principle of scoping each node to only the tools/data its job needs comes from the role-design thinking behind the Day 4 CrewAI crew, applied here as a design rule rather than a separate framework.




## 3. Framework Choice Rationale

LangGraph was chosen over CrewAI because this is a control-heavy pipeline, not a role-negotiation problem: there is one deterministic sequence with explicit branches (bad input, refusal, needs-approval, quality-retry, tool-timeout), and every branch needs to be inspectable and resumable — LangGraph's explicit state graph and conditional edges give us that directly, including a natural pause point for the human-approval checkpoint. CrewAI's strength is autonomous role-to-role collaboration (e.g. a researcher, writer, and editor negotiating a shared output), which isn't the shape of this problem. A raw while-loop could implement the same logic but would lose the typed state, the visual/auditable graph, and the built-in conditional routing LangGraph provides for compliance-sensitive workflows like this one.

## 4. Evaluation Results

Ten test cases were run (7 core scenarios plus 3 edge/adversarial cases: an empty-input case, a prompt-injection attempt, and an oversized 5,000-character input) against six criteria: task success, factual accuracy, safety, latency, tone quality, and graceful failure handling (1–5 scale; latency in ms, informational).

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| TC1 | Delivery timeline question | sent | sent | 5 | 5 | 5 | 6.93 | 5 | 5 |
| TC2 | Refund request | sent | sent | 5 | 5 | 5 | 5.93 | 5 | 5 |
| TC3 | Contract cancel | rejected | rejected | 5 | 5 | 5 | 6.28 | 5 | 5 |
| TC4 | Smart contract exploit | sent | sent | 5 | 5 | 5 | 5.64 | 5 | 5 |
| TC5 | Onboarding question | sent | sent | 5 | 5 | 5 | 4.01 | 5 | 5 |
| TC6 | Billing dispute | sent | sent | 5 | 5 | 5 | 6.03 | 5 | 5 |
| TC7 | Gibberish/no-KB-match | sent | sent (retry=1, score 60→85) | 5 | 5 | 5 | 7.53 | 5 | 5 |
| TC8 | Empty input | rejected_invalid_input | rejected (HTTP 422) | 5 | 5 | 4 | 2.13 | 4 | 5 |
| TC9 | Prompt injection | refused | refused | 5 | 5 | 4 | 3.72 | 4 | 5 |
| TC10 | Oversized (5000 chars) | rejected_invalid_input | rejected_invalid_input | 5 | 5 | 4 | 2.67 | 4 | 5 |

**Result:** 10/10 test cases reached the expected terminal status. All consequential-action tickets (TC2, TC3, TC4, TC6) were correctly routed through the human-approval checkpoint. The prompt-injection case (TC9) was correctly refused. Both edge cases (empty input, 5,000-character input) were rejected cleanly rather than crashing the pipeline. A separate induced-failure run (forcing the outbound send tool to time out) confirmed the agent degrades to a failed_will_retry status instead of throwing an unhandled exception.

**Live API verification:** In addition to the offline evaluation above, the deployed FastAPI service was tested live against 6 representative tickets via POST /triage. All 6 responded correctly:

- Routine timeline ticket → sent (P3-normal, needs_approval: false)
- Refund request → awaiting_human_approval (P1-critical, needs_approval: true)
- Prompt-injection attempt → refused
- Contract cancellation → awaiting_human_approval (P2-high, needs_approval: true)
- Smart-contract bug report → awaiting_human_approval (P1-critical, needs_approval: true)
- Empty input → rejected at the API layer with HTTP 422 (string_too_short, min_length=1), proving input validation runs before the agent graph — a defense-in-depth pattern.

### Most Common Failure Pattern (and How the Self-Correction Loop Addresses It)

The recurring weakness is **generic escalation drafting**: when a ticket doesn't match any KB article (TC7), the first draft is a generic "looping in a specialist" line regardless of what the client actually asked. The critique_draft node catches this itself — it scores that first draft 60/100 (below the 80 threshold) specifically for being non-specific, which triggers the reused Day-3 revise loop. The regenerated draft echoes the client's own question back, and the critique score rises to 85. This is a genuine case of the self-correction loop fixing a real weakness, not just retrying blindly — confirmed in evaluate.py's TC7 run (retries=1, critique_score=85).

**Residual limitation:** this only fixes specificity, not accuracy — KB matching is still exact-keyword, so a paraphrased question can still miss a real KB article on the first pass. **Fix:** replace the keyword matcher with embedding-based semantic search over a larger KB.

---

## 5. Known Limitations

- classify_ticket and draft_response are deterministic stand-ins for an LLM call (by design, to keep the capstone runnable with zero API keys) — real client phrasing will be far more varied than keyword rules can cover.
- KB matching is exact-keyword, not semantic — paraphrased questions can miss a real match (see failure pattern above).
- The human-approval queue uses LangGraph's InMemorySaver checkpointer; it does not survive a restart and won't scale across multiple workers (swap in SqliteSaver/PostgresSaver for that, no other code changes needed).
- No real outbound integration (email / Zendesk / Intercom) yet — send_response is a mock.

---

## 6. Recommended Next Steps

- **Scaling:** Swap InMemorySaver for SqliteSaver/PostgresSaver so the approval queue survives restarts; swap classify_ticket/generate_draft/critique_draft for real Claude calls with the same function signatures; add semantic KB search.
- **Guardrails:** Add a second-pass safety check on drafts before send (PII leakage, promise-not-to-make claims like guaranteed refund amounts); expand the refusal trigger list from keyword-based to classifier-based.
- **Human oversight:** Build a lightweight approval-queue UI (the API already exposes the resume endpoint); track approval turnaround time as a new monitoring metric; review a sample of auto-sent (non-gated) tickets weekly.
- **Monitoring:** Wire the structured logs (logs/agent.log) into a dashboard; alert on error-rate spikes, latency regressions, and a rising proportion of no-KB-match tickets (a proxy for KB coverage drift).